🐳 Atelier 4 — Docker Compose Multi-Services : La Stack Complète

---

🎯 Scénario réel (accroche)

> *"Une vraie application en production, ce n'est jamais juste un fichier Python. C'est une API, une base de données, un cache, un reverse proxy — tout connecté, tout orchestré, tout déployé en une seule commande."*

Ce qu'on va construire : une stack complète avec 4 services qui communiquent entre eux, protégée par un pipeline CI/CD complet.

🗺️ Vision globale de la stack

```
                    Internet
                       |
                    [Port 80]
                       |
              ┌────────▼────────┐
              │      NGINX      │  ← Reverse Proxy
              │  (nginx:alpine) │     Reçoit tout le trafic
              └─────────────────┘
                       |
              ┌────────▼────────┐
              │   FLASK API     │  ← Notre application Python
              │  (notre image)  │     Port 5000 (interne)
              └─────────────────┘
                 |          |
     ┌──────────▼──┐   ┌────▼──────────┐
     │  POSTGRESQL  │   │     REDIS     │
     │  (postgres)  │   │ (redis:alpine) │
     │  Base données │   │ Cache & sessions│
     │  Port 5432   │   │ Port 6379     │
     └──────────────┘   └───────────────┘
```

📁 Structure complète du projet

```
stack-complete/
├── api/
│   ├── app.py                  ← Application Flask
│   ├── database.py             ← Connexion PostgreSQL
│   ├── cache.py                ← Connexion Redis
│   └── routes/
│       ├── __init__.py
│       ├── health.py           ← Routes de santé
│       ├── users.py            ← Routes métier
│       └── stats.py            ← Routes stats (cache Redis)
├── tests/
│   ├── __init__.py
│   ├── conftest.py             ← Fixtures pytest
│   ├── test_health.py
│   ├── test_users.py
│   └── test_stats.py
├── nginx/
│   └── nginx.conf              ← Config reverse proxy
├── postgres/
│   └── init.sql                ← Script d'initialisation BDD
├── Dockerfile
├── .dockerignore
├── requirements.txt
├── docker-compose.yml          ← Dev local (4 services)
├── docker-compose.prod.yml     ← Production
├── docker-compose.test.yml     ← Tests intégration
├── .env.example
├── .gitignore
└── .github/
    └── workflows/
        ├── ci.yml              ← Tests unitaires + intégration
        └── cd.yml              ← Build + Push + Deploy
```

🏗️ PARTIE 1 — Créer le repo et la structure

Étape 1 — Créer le repo GitHub

Sur github.com → **"New repository"**

Champ	Valeur
Repository name	`stack-complete`
Visibility	Public
Initialize with README	✅ Cocher

Étape 2 — Cloner et créer la structure

```
git clone https://github.com/TON_USERNAME/stack-complete.git
cd stack-complete

# Créer tous les dossiers
mkdir -p api/routes tests nginx postgres .github/workflows

# Créer tous les fichiers
touch api/app.py api/database.py api/cache.py
touch api/routes/__init__.py api/routes/health.py
touch api/routes/users.py api/routes/stats.py
touch tests/__init__.py tests/conftest.py
touch tests/test_health.py tests/test_users.py tests/test_stats.py
touch nginx/nginx.conf
touch postgres/init.sql
touch Dockerfile .dockerignore requirements.txt
touch docker-compose.yml docker-compose.prod.yml docker-compose.test.yml
touch .env.example .gitignore
touch .github/workflows/ci.yml .github/workflows/cd.yml
```

📝 PARTIE 2 — Écrire l'application Python

Étape 3 — Connexion PostgreSQL

`api/database.py`

```
import os
import psycopg2
from psycopg2.extras import RealDictCursor

DATABASE_URL = os.getenv("DATABASE_URL",
    "postgresql://user:password@localhost:5432/appdb")
```

```python
def get_connection():
    return psycopg2.connect(DATABASE_URL, cursor_factory=RealDictCursor)

def init_db():
    """Crée les tables si elles n'existent pas"""
    conn = get_connection()
    cur  = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id         SERIAL PRIMARY KEY,
            name       VARCHAR(100) NOT NULL,
            email      VARCHAR(100) UNIQUE NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    cur.close()
    conn.close()

def get_all_users():
    conn  = get_connection()
    cur   = conn.cursor()
    cur.execute("SELECT * FROM users ORDER BY created_at DESC")
    users = cur.fetchall()
    cur.close()
    conn.close()
    return [dict(u) for u in users]

def create_user(name, email):
    conn = get_connection()
    cur  = conn.cursor()
    cur.execute(
        "INSERT INTO users (name, email) VALUES (%s, %s) RETURNING *",
        (name, email)
    )
    user = dict(cur.fetchone())
    conn.commit()
    cur.close()
    conn.close()
    return user
```

Étape 4 — Connexion Redis

`api/cache.py`

```python
import os
import redis
import json

REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")

def get_redis():
    return redis.from_url(REDIS_URL, decode_responses=True)

def get_cached(key):
    """Récupère une valeur du cache"""
    r   = get_redis()
    val = r.get(key)
    return json.loads(val) if val else None

def set_cached(key, value, ttl=60):
    """Met en cache avec expiration (TTL en secondes)"""
    r = get_redis()
    r.setex(key, ttl, json.dumps(value))

def invalidate(key):
    """Supprime une entrée du cache"""
    r = get_redis()
    r.delete(key)

def get_stats():
    """Statistiques Redis pour le monitoring"""
    r    = get_redis()
    info = r.info()
    return {
        "connected_clients" : info["connected_clients"],
        "used_memory_human" : info["used_memory_human"],
        "keyspace_hits"     : info.get("keyspace_hits",   0),
        "keyspace_misses"   : info.get("keyspace_misses", 0)
    }
```

Étape 5 — Les routes

`api/routes/health.py`

```python
from flask import Blueprint, jsonify
import os

health_bp = Blueprint("health", __name__)

@health_bp.route("/health")
def health():
    return jsonify({
        "status"      : "ok",
        "environment" : os.getenv("APP_ENV", "local"),
        "version"     : os.getenv("APP_VERSION", "1.0.0")
    })

@health_bp.route("/ready")
def ready():
    """Vérifie que tous les services sont disponibles"""
    from api.database import get_connection
    from api.cache    import get_redis
    checks = {}

    # Vérif PostgreSQL
    try:
        conn = get_connection()
        conn.close()
        checks["postgres"] = "ok"
    except Exception as e:
        checks["postgres"] = f"error: {str(e)}"

    # Vérif Redis
    try:
        r = get_redis()
        r.ping()
        checks["redis"] = "ok"
    except Exception as e:
        checks["redis"] = f"error: {str(e)}"

    all_ok = all(v == "ok" for v in checks.values())
    return jsonify({
        "status" : "ready" if all_ok else "degraded",
        "checks" : checks
    }), 200 if all_ok else 503
```

---

api/routes/users.py

```python
from flask import Blueprint, jsonify, request
from api.database import get_all_users, create_user
from api.cache    import get_cached, set_cached, invalidate

users_bp = Blueprint("users", __name__)

@users_bp.route("/users", methods=["GET"])
def list_users():
    # Vérifier le cache d'abord
    cached = get_cached("users:all")
    if cached:
        return jsonify({"source": "cache", "data": cached})

    # Sinon aller en base
    users = get_all_users()
    set_cached("users:all", users, ttl=30)
    return jsonify({"source": "database", "data": users})

@users_bp.route("/users", methods=["POST"])
def add_user():
    body  = request.get_json()
    name  = body.get("name")
    email = body.get("email")

    if not name or not email:
        return jsonify({"error": "name et email requis"}), 400

    user = create_user(name, email)
    invalidate("users:all")    # invalider le cache après écriture
    return jsonify(user), 201
```

---

api/routes/stats.py

```python
from flask import Blueprint, jsonify
from api.cache import get_stats

stats_bp = Blueprint("stats", __name__)

@stats_bp.route("/stats")
def stats():
    return jsonify({
```

```
        "cache" : get_stats()
    })
```

Étape 6 — L'application principale

`api/app.py`

```
from flask import Flask
import os

def create_app():
    app = Flask(__name__)

    # Enregistrer les blueprints
    from api.routes.health import health_bp
    from api.routes.users  import users_bp
    from api.routes.stats  import stats_bp

    app.register_blueprint(health_bp)
    app.register_blueprint(users_bp)
    app.register_blueprint(stats_bp)

    return app

app = create_app()

if __name__ == "__main__":
    from api.database import init_db
    init_db()                               # créer les tables au démarrage
    port = int(os.getenv("PORT", 5000))
    app.run(host="0.0.0.0", port=port)
```

Étape 7 — Les dépendances

`requirements.txt`

```
flask==3.0.3
psycopg2-binary==2.9.9
redis==5.0.4
```

```
pytest==8.2.0
pytest-cov==5.0.0
```

🧪 PARTIE 3 — Les tests

Étape 8 — Fixtures pytest

`tests/conftest.py`

```python
import pytest
from unittest.mock import patch, MagicMock
from api.app import create_app

@pytest.fixture
def app():
    app = create_app()
    app.config["TESTING"] = True
    return app

@pytest.fixture
def client(app):
    return app.test_client()

@pytest.fixture
def mock_db():
    """Mock PostgreSQL - pas besoin de vraie BDD pour les tests unitaires"""
    with patch("api.routes.users.get_all_users") as mock_get, \
         patch("api.routes.users.create_user")   as mock_create:
        yield mock_get, mock_create

@pytest.fixture
def mock_cache():
    """Mock Redis - pas besoin de vrai Redis pour les tests unitaires"""
    with patch("api.routes.users.get_cached")  as mock_get, \
         patch("api.routes.users.set_cached")  as mock_set, \
         patch("api.routes.users.invalidate")  as mock_inv:
        yield mock_get, mock_set, mock_inv
```

`tests/test_health.py`

```python
def test_health(client):
    r = client.get("/health")
    assert r.status_code == 200
    data = r.get_json()
    assert data["status"] == "ok"
    assert "environment" in data
    assert "version"     in data

def test_ready_avec_services_ok(client):
    from unittest.mock import patch, MagicMock

    mock_conn  = MagicMock()
    mock_redis = MagicMock()
    mock_redis.ping.return_value = True

    with patch("api.routes.health.get_connection", return_value=mock_conn), \
         patch("api.routes.health.get_redis",      return_value=mock_redis):
        r = client.get("/ready")
        assert r.status_code == 200
        data = r.get_json()
        assert data["status"]           == "ready"
        assert data["checks"]["postgres"] == "ok"
        assert data["checks"]["redis"]    == "ok"
```

---

tests/test_users.py

```python
def test_list_users_depuis_cache(client, mock_db, mock_cache):
    mock_get_cached, _, _ = mock_cache
    mock_get_cached.return_value = [
        {"id": 1, "name": "Alice", "email": "alice@test.com"}
    ]

    r = client.get("/users")
    assert r.status_code == 200
    data = r.get_json()
    assert data["source"] == "cache"          # vient du cache
    assert len(data["data"]) == 1

def test_list_users_depuis_bdd(client, mock_db, mock_cache):
    mock_get_cached, mock_set_cached, _ = mock_cache
    mock_get_users, _                   = mock_db

    mock_get_cached.return_value = None       # cache vide
```

```
    mock_get_users.return_value  = [
        {"id": 1, "name": "Bob", "email": "bob@test.com"}
    ]

    r = client.get("/users")
    assert r.status_code == 200
    data = r.get_json()
    assert data["source"] == "database"       # vient de la BDD

def test_create_user(client, mock_db, mock_cache):
    _, mock_create_user         = mock_db
    _, _, mock_invalidate       = mock_cache
    mock_create_user.return_value = {
        "id": 2, "name": "Carol", "email": "carol@test.com"
    }

    r = client.post("/users", json={
        "name" : "Carol",
        "email": "carol@test.com"
    })
    assert r.status_code == 201
    mock_invalidate.assert_called_once_with("users:all")  # cache invalidé

def test_create_user_champs_manquants(client):
    r = client.post("/users", json={"name": "Dave"})
    assert r.status_code == 400
    assert "error" in r.get_json()
```

🐳 PARTIE 4 — Le Dockerfile

Étape 9 — Dockerfile optimisé multi-stage

`Dockerfile`

```
# ─────────────────────────────────────────────
# STAGE 1 : Builder (installe les dépendances)
# ─────────────────────────────────────────────
FROM python:3.11-slim AS builder

WORKDIR /app

# Copier et installer les dépendances
COPY requirements.txt .
```

```dockerfile
RUN pip install --no-cache-dir --user -r requirements.txt

# ──────────────────────────────────────────────
# STAGE 2 : Production (image finale, légère)
# ──────────────────────────────────────────────
FROM python:3.11-slim AS production

WORKDIR /app

# Copier uniquement les dépendances installées
COPY --from=builder /root/.local /root/.local

# Copier le code de l'application
COPY api/       ./api/
COPY postgres/  ./postgres/

# Variables d'environnement
ENV PATH=/root/.local/bin:$PATH
ENV APP_ENV=production
ENV APP_VERSION=1.0.0
ENV PYTHONPATH=/app

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:5000/health')"

CMD ["python", "-m", "api.app"]
```

> **Multi-stage build :** l'image finale ne contient pas les outils de build — elle est plus légère et plus sécurisée. C'est la pratique standard en production.

`.dockerignore`

```
.git
.github
tests/
*.md
docs/
.env
.env.*
__pycache__/
```

```
*.pyc
.coverage
htmlcov/
.pytest_cache/
venv/
.venv/
```

🔧 PARTIE 5 — Docker Compose

Étape 10 — Variables d'environnement

`.env.example`

```
# Application
APP_ENV=development
APP_VERSION=1.0.0
PORT=5000

# PostgreSQL
POSTGRES_DB=appdb
POSTGRES_USER=user
POSTGRES_PASSWORD=password
DATABASE_URL=postgresql://user:password@postgres:5432/appdb

# Redis
REDIS_URL=redis://redis:6379/0

# GitHub (pour la prod)
GITHUB_USERNAME=ton_username
```

```
# En local
cp .env.example .env
```

```
# .gitignore
echo ".env" >> .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore
```

Étape 11 — Docker Compose développement

`docker-compose.yml`

```yaml
# ──────────────────────────────────────────────────────────
# DÉVELOPPEMENT LOCAL — 4 services complets
# Usage : docker compose up
# ──────────────────────────────────────────────────────────

version: "3.9"

services:

  # ── 1. PostgreSQL ──────────────────────────────────
  postgres:
    image: postgres:15-alpine
    container_name: stack-postgres

    environment:
      POSTGRES_DB      : ${POSTGRES_DB}
      POSTGRES_USER    : ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

    ports:
      - "5432:5432"       # accessible en local pour debug

    volumes:
      - postgres_data:/var/lib/postgresql/data    # persistance des données
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql

    healthcheck:
      test    : ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
${POSTGRES_DB}"]
      interval: 10s
      timeout : 5s
      retries : 5

    restart: unless-stopped

  # ── 2. Redis ───────────────────────────────────────
  redis:
    image: redis:7-alpine
    container_name: stack-redis

    ports:
      - "6379:6379"       # accessible en local pour debug

    volumes:
```

```yaml
      - redis_data:/data

    healthcheck:
      test    : ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout : 5s
      retries : 5

    restart: unless-stopped

  # ── 3. Flask API ──────────────────────────────
  app:
    build:
      context   : .
      dockerfile: Dockerfile
      target    : production

    container_name: stack-api

    env_file: .env

    environment:
      APP_ENV     : development
      DATABASE_URL:
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB
}
      REDIS_URL   : redis://redis:6379/0

    ports:
      - "5000:5000"

    volumes:
      - ./api:/app/api    # rechargement à chaud en dev

    depends_on:
      postgres:
        condition: service_healthy    # attend PostgreSQL
      redis:
        condition: service_healthy    # attend Redis

    healthcheck:
      test    : ["CMD-SHELL", "curl -f http://localhost:5000/health || exit
1"]
      interval: 15s
      timeout : 10s
      retries : 3
```

```yaml
    restart: unless-stopped

  # ── 4. Nginx ──────────────────────────────────────
  nginx:
    image: nginx:alpine
    container_name: stack-nginx

    ports:
      - "80:80"           # point d'entrée unique

    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro

    depends_on:
      app:
        condition: service_healthy    # attend Flask

    restart: unless-stopped

# ── Volumes persistants ──────────────────────────
volumes:
  postgres_data:    # données PostgreSQL survivent aux restarts
  redis_data:       # données Redis survivent aux restarts
```

Étape 12 — Docker Compose tests d'intégration

`docker-compose.test.yml`

```yaml
# ──────────────────────────────────────────────────────
# TESTS D'INTÉGRATION — services réels, pas de mocks
# Usage : docker compose -f docker-compose.test.yml up --abort-on-container-
exit
# ──────────────────────────────────────────────────────

version: "3.9"

services:

  postgres-test:
    image: postgres:15-alpine
    container_name: test-postgres
    environment:
```

```
      POSTGRES_DB      : testdb
      POSTGRES_USER    : testuser
      POSTGRES_PASSWORD: testpass
    healthcheck:
      test    : ["CMD-SHELL", "pg_isready -U testuser -d testdb"]
      interval: 5s
      timeout : 3s
      retries : 10

  redis-test:
    image: redis:7-alpine
    container_name: test-redis
    healthcheck:
      test    : ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout : 3s
      retries : 5

  test-runner:
    build:
      context   : .
      dockerfile: Dockerfile
    container_name: test-runner
    environment:
      APP_ENV     : test
      DATABASE_URL: postgresql://testuser:testpass@postgres-test:5432/testdb
      REDIS_URL   : redis://redis-test:6379/0
    depends_on:
      postgres-test:
        condition: service_healthy
      redis-test:
        condition: service_healthy
    command: >
      sh -c "pip install pytest pytest-cov &&
             pytest tests/ --cov=api --cov-report=term-missing --cov-fail-
under=80"
    profiles: ["test"]
```

Étape 13 — Docker Compose production

`docker-compose.prod.yml`

```yaml
# ──────────────────────────────────────────────────────────
# PRODUCTION — image depuis le registry
# Usage : docker compose -f docker-compose.prod.yml up -d
# ──────────────────────────────────────────────────────────

version: "3.9"

services:

  postgres:
    image: postgres:15-alpine
    container_name: prod-postgres
    environment:
      POSTGRES_DB      : ${POSTGRES_DB}
      POSTGRES_USER    : ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_prod:/var/lib/postgresql/data
    healthcheck:
      test    : ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
${POSTGRES_DB}"]
      interval: 10s
      timeout : 5s
      retries : 5
    restart: always

  redis:
    image: redis:7-alpine
    container_name: prod-redis
    volumes:
      - redis_prod:/data
    healthcheck:
      test    : ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout : 5s
      retries : 5
    restart: always

  app:
    image: ghcr.io/${GITHUB_USERNAME}/stack-complete:latest    # image depuis
registry
    container_name: prod-api
    environment:
      APP_ENV     : production
      APP_VERSION : ${APP_VERSION}
      DATABASE_URL:
```

```
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB
}
      REDIS_URL   : redis://redis:6379/0
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    healthcheck:
      test    : ["CMD-SHELL", "curl -f http://localhost:5000/health || exit
1"]
      interval: 15s
      timeout : 10s
      retries : 3
    restart: always

  nginx:
    image: nginx:alpine
    container_name: prod-nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      app:
        condition: service_healthy
    restart: always

volumes:
  postgres_prod:
  redis_prod:
```

Étape 14 — Configuration Nginx

`nginx/nginx.conf`

```
upstream flask_api {
    server app:5000;
}

server {
    listen 80;
```

```
    # ── Routes de l'API ──────────────────────────
    location / {
        proxy_pass         http://flask_api;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;

        # Timeouts
        proxy_connect_timeout 30s;
        proxy_read_timeout    30s;
    }

    # ── Health check (pas de log) ────────────────
    location /health {
        proxy_pass http://flask_api;
        access_log off;
    }

    # ── Readiness check ──────────────────────────
    location /ready {
        proxy_pass http://flask_api;
        access_log off;
    }
}
```

Étape 15 — Init SQL

`postgres/init.sql`

```sql
-- Créé automatiquement au premier démarrage de PostgreSQL

CREATE TABLE IF NOT EXISTS users (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Données de démonstration
INSERT INTO users (name, email) VALUES
    ('Alice Dupont', 'alice@example.com'),
    ('Bob Martin',   'bob@example.com')
ON CONFLICT DO NOTHING;
```

⚙️ PARTIE 6 — Les workflows GitHub Actions

Étape 16 — CI complet

`.github/workflows/ci.yml`

```yaml
name: CI - Tests unitaires et intégration

on:
  pull_request:
    branches: [ main ]

jobs:

  # ── Tests unitaires (rapides, sans services) ──
  unit-tests:
    name: ✅ Tests unitaires
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - run: pip install -r requirements.txt

      - name: Tests unitaires avec mocks
        run: |
          pytest tests/test_health.py tests/test_users.py tests/test_stats.py
\
            --cov=api \
            --cov-report=term-missing \
            --cov-fail-under=80
        env:
          PYTHONPATH: .

  # ── Tests d'intégration (avec vrais services) ──
  integration-tests:
    name: 🔗 Tests d'intégration
    runs-on: ubuntu-latest
    needs: unit-tests           # seulement si les tests unitaires passent
```

```yaml
    steps:
      - uses: actions/checkout@v4

      - name: Créer le fichier .env de test
        run: |
          echo "POSTGRES_DB=testdb"         > .env
          echo "POSTGRES_USER=testuser"    >> .env
          echo "POSTGRES_PASSWORD=testpass">> .env

      - name: Lancer les services de test
        run: |
          docker compose -f docker-compose.test.yml \
            --profile test up \
            --abort-on-container-exit \
            --exit-code-from test-runner

      - name: Nettoyage
        if: always()
        run: docker compose -f docker-compose.test.yml down -v
```

Étape 17 — CD complet

`.github/workflows/cd.yml`

```yaml
name: CD - Build, Push et Deploy Stack Complète

on:
  push:
    branches: [ main ]

env:
  REGISTRY  : ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:

  # ── Tests avant tout ──────────────────────────
  test:
    name: ✅ Tests
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
```

```yaml
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: pytest --cov=api --cov-fail-under=80
        env:
          PYTHONPATH: .

  # ── Build et Push de l'image ──────────────────
  build-and-push:
    name: 🐳 Build & Push
    runs-on: ubuntu-latest
    needs: test

    permissions:
      contents: read
      packages: write

    outputs:
      image-tag: ${{ steps.meta.outputs.version }}

    steps:
      - uses: actions/checkout@v4

      - name: Connexion à GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extraction des métadonnées
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=sha,prefix=sha-
            type=raw,value=latest
            type=semver,pattern={{version}}

      - name: Build et Push
        uses: docker/build-push-action@v5
        with:
          context   : .
          target    : production
```

```yaml
          push      : true
          tags      : ${{ steps.meta.outputs.tags }}
          labels    : ${{ steps.meta.outputs.labels }}

      - name: Résumé du build
        run: |
          echo "## 🐳 Image publiée" >> $GITHUB_STEP_SUMMARY
          echo "Tags : ${{ steps.meta.outputs.tags }}" >> $GITHUB_STEP_SUMMARY

  # ── Déploiement de la stack complète ──────────
  deploy:
    name: 🚀 Deploy Stack Complète
    runs-on: ubuntu-latest
    needs: build-and-push

    environment:
      name: production

    steps:
      - uses: actions/checkout@v4

      - name: Connexion au registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Création du .env de production
        run: |
          echo "APP_ENV=production"                          > .env
          echo "APP_VERSION=${{ github.sha }}"             >> .env
          echo "GITHUB_USERNAME=${{ github.actor }}"       >> .env
          echo "POSTGRES_DB=${{ secrets.POSTGRES_DB }}"    >> .env
          echo "POSTGRES_USER=${{ secrets.POSTGRES_USER }}">> .env
          echo "POSTGRES_PASSWORD=${{ secrets.POSTGRES_PASSWORD }}" >> .env

      - name: Pull des nouvelles images
        run: docker compose -f docker-compose.prod.yml pull

      - name: Déploiement de la stack complète
        run: |
          docker compose -f docker-compose.prod.yml up -d \
            --remove-orphans \
            --wait              # attend que tous les healthchecks passent
```

```
      - name: Vérification de la stack
        run: |
          echo "── Services actifs ──"
          docker compose -f docker-compose.prod.yml ps

          echo "── Test endpoint /ready ──"
          sleep 5
          curl -f http://localhost/ready

          echo "✅ Stack complète déployée et opérationnelle !"

      - name: Résumé du déploiement
        run: |
          echo "## 🚀 Stack déployée" >> $GITHUB_STEP_SUMMARY
          echo "- Version : ${{ github.sha }}" >> $GITHUB_STEP_SUMMARY
          echo "- Services : nginx + flask + postgres + redis" >>
$GITHUB_STEP_SUMMARY
```

🔑 PARTIE 7 — Configurer les secrets GitHub

Étape 18 — Ajouter les secrets

Sur GitHub → Settings → **Secrets and variables** → **Actions**

Créer ces 3 secrets :

Name	Valeur
`POSTGRES_DB`	`appdb`
`POSTGRES_USER`	`user`
`POSTGRES_PASSWORD`	*(mot de passe fort)*

Activer les droits d'écriture :
GitHub → Settings → Actions → General → **"Read and write permissions"** ✅

🚀 PARTIE 8 — Tester en local puis pousser

Étape 19 — Démarrer la stack en local

```
# Démarrer les 4 services
docker compose up --build
```

Sortie attendue :

```
✓ Container stack-postgres  Healthy
✓ Container stack-redis     Healthy
✓ Container stack-api       Healthy
✓ Container stack-nginx     Started
```

Étape 20 — Tester tous les endpoints

```
# Via Nginx (port 80) — comme en production
curl http://localhost/health
curl http://localhost/ready
curl http://localhost/users
curl http://localhost/stats

# Créer un utilisateur
curl -X POST http://localhost/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@test.com"}'

# Relister — source: database
curl http://localhost/users

# Relister — source: cache (2e appel)
curl http://localhost/users
```

Réponses :

```
// GET /ready
{
  "status": "ready",
  "checks": {
    "postgres": "ok",
    "redis"   : "ok"
  }
}

// GET /users (1er appel)
```

```
{ "source": "database", "data": [...] }

// GET /users (2e appel)
{ "source": "cache", "data": [...] }
```

Étape 21 — Observer les logs de tous les services

```
# Tous les services
docker compose logs -f

# Un service spécifique
docker compose logs -f app
docker compose logs -f postgres
docker compose logs -f nginx
```

Étape 22 — Premier push et pipeline complet

```
git add .
git commit -m "feat: stack complète Flask + PostgreSQL + Redis + Nginx"
git push origin main
```

Pipeline GitHub Actions :

```
🟡 CD - Build, Push et Deploy Stack Complète
   │
   ├── ✅ Tests              → 12 tests passés
   │
   ├── ✅ Build & Push
   │      └── ghcr.io/tonuser/stack-complete:sha-9fb40e3
   │      └── ghcr.io/tonuser/stack-complete:latest
   │
   └── ✅ Deploy Stack Complète
         ├── postgres  → Healthy ✅
         ├── redis     → Healthy ✅
         ├── app       → Healthy ✅
         ├── nginx     → Started ✅
         └── /ready    → {"status": "ready"} ✅
```

🗺️ Récapitulatif visuel du flux complet

```
[Dev local]
     |
     |── docker compose up ──► 4 services démarrés
     |                         postgres + redis + flask + nginx
     |
     |── Tests locaux ────────► pytest (mocks)
     |
     |── git push ────────────► feature/xxx
                                     |
                              [Pull Request]
                                     |
                              ✅ Tests unitaires
                              ✅ Tests d'intégration
                                (vrais services Docker)
                                     |
                              🔀 Merge → main
                                     |
                              🐳 Build image multi-stage
                              📦 Push → ghcr.io:sha-xxx
                                     |
                              🚀 docker compose prod up
                                ├── postgres (volume persistant)
                                ├── redis    (volume persistant)
                                ├── flask    (image registry)
                                └── nginx    (reverse proxy)
                                     |
                              ✅ /ready → tout vert
```

🎓 Débrief final (20 min)

Question	Ce qu'on veut entendre
Pourquoi `depends_on` avec `service_healthy` ?	Évite que Flask démarre avant que PostgreSQL soit prêt
Pourquoi des volumes nommés ?	Les données survivent aux `docker compose down`
Quelle différence entre tests unitaires et intégration ?	Unitaires = mocks rapides / Intégration = vrais services

Question	Ce qu'on veut entendre
Pourquoi invalider le cache après un POST ?	Sinon le GET suivant retourne des données périmées
Comment rollback si la prod est cassée ?	Changer le tag de l'image et `docker compose up -d`
Pourquoi multi-stage Dockerfile ?	Image finale légère, sans outils de build

---

📚 Vocabulaire installé

Terme	Définition vécue
Service	Un container défini dans Compose — postgres, redis, app, nginx
Volume nommé	Stockage persistant géré par Docker — survit aux redémarrages
depends_on + healthy	Ordre de démarrage garanti — personne ne démarre avant que son dépendant soit prêt
Cache invalidation	Supprimer le cache quand les données changent — cohérence garantie
Multi-stage build	Dockerfile en deux passes — image finale allégée
Readiness probe	Endpoint `/ready` qui vérifie que tous les services répondent

---

🚀 Progression complète des 4 ateliers

```
Atelier 1  →  CI        Bloquer le code cassé avant le merge
Atelier 2  →  CD        Livrer automatiquement sur Vercel
Atelier 3  →  Docker    Builder, publier, déployer un container
Atelier 4  →  Stack     Orchestrer plusieurs services en production  ← (cet
atelier)
──────────────────────────────────────────────────────────────────────────
Atelier 5  →  Monitoring  Savoir quand la prod est cassée
               Sentry + alertes + rollback automatique
```